

UNIVERSIDAD PERUANA LOS ANDES

FACULTAD DE INGENIERÍA

**ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS Y
COMPUTACIÓN**



TEMA:

EVIDENCIA SEMANA 01 ACTIVIDAD 01 - 2026

CURSO:

ALGORITMOS Y ESTRUCTURA DE DATOS

CICLO:

V

ALUMNO:

TAIPE SULLA, FERNANDO

HUANCAYO – PERU

2026

Arreglos en Java con programación orientada a objetos

Ocho operaciones sobre un arreglo de enteros, con el código, las evidencias de ejecución, los resultados y la explicación paso a paso.

1. Introducción

Este documento presenta el desarrollo de la Actividad 1 sobre arreglos. El programa está hecho en Java con NetBeans y maneja un arreglo de enteros de tamaño fijo desde un menú de consola. Sobre ese arreglo se implementaron ocho operaciones: eliminar, modificar, insertar, sumar, calcular menor, mayor y promedio, sumar impares y dos movimientos de elementos.

El código se organizó con los cuatro pilares de la programación orientada a objetos: encapsulamiento, herencia, abstracción y polimorfismo. Para cada actividad se incluye el análisis, el algoritmo, el código, su explicación, la evidencia de ejecución y los resultados obtenidos.

Objetivos

Manipular un arreglo unidimensional de enteros con inserción, eliminación, modificación y desplazamiento de elementos.

Aplicar recorridos con acumuladores y comparaciones para obtener suma, menor, mayor, promedio y suma de impares.

Estructurar la solución con clases, herencia, una clase abstracta, una interfaz y polimorfismo.

Herramientas

Java y NetBeans. El proyecto está configurado para JDK 26; el código solo usa características disponibles desde Java 8.

2. Enunciado de las actividades

Elaborar un método para eliminar un elemento "x", del arreglo creado anteriormente.

Elaborar un método para modificar un elemento "x".

Elaborar un método para insertar un elemento "x" en una posición "y".

Elaborar un método para sumar los elementos del arreglo.

Elaborar un método para hallar el número menor, el número mayor y el promedio de los elementos del arreglo.

Elaborar un método para suma de impares de los elementos del arreglo.

Elaborar un método para mover el primer elemento a la última posición del arreglo.

Elaborar un método para mover el primer elemento par al final del arreglo.

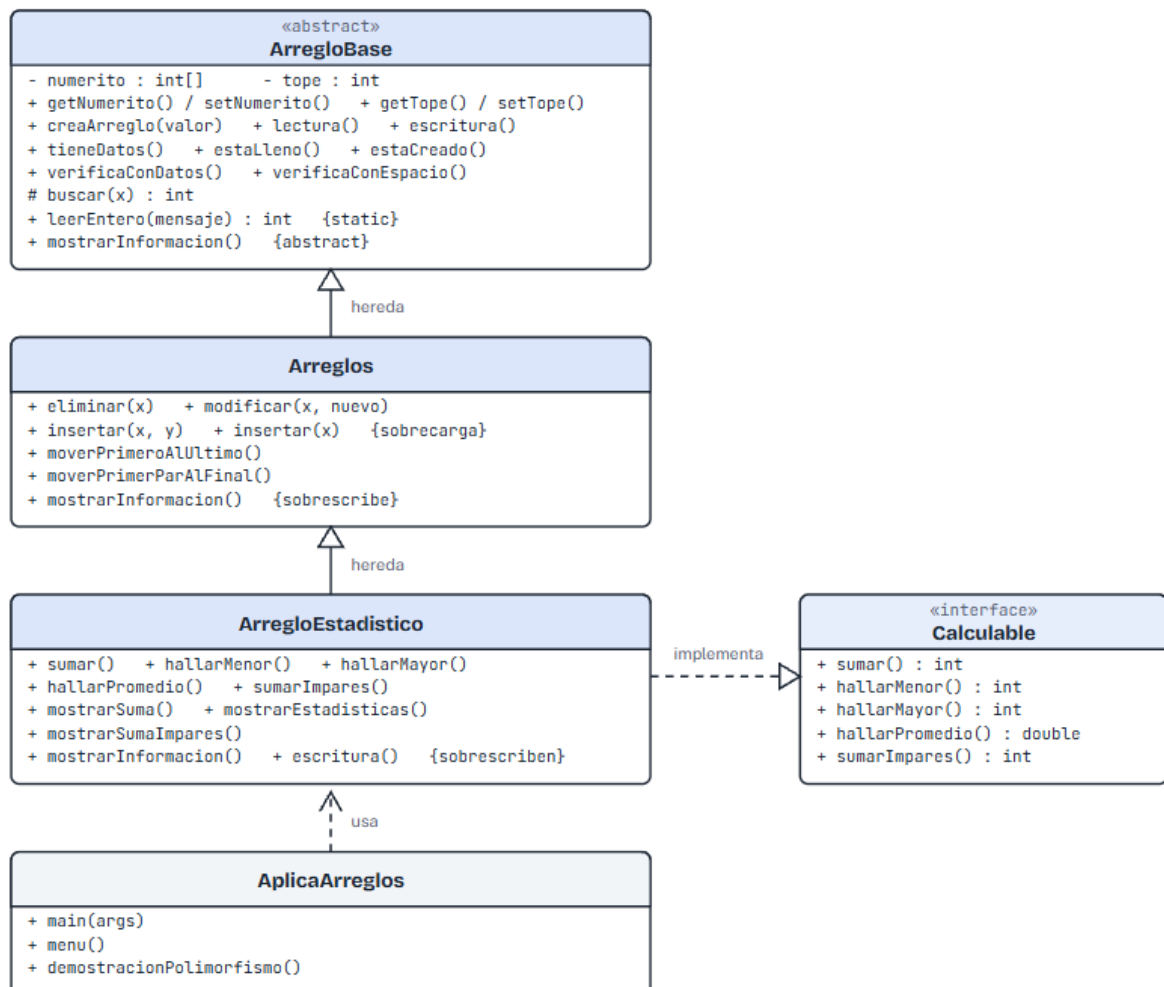
Requisito adicional: la solución debe aplicar programación orientada a objetos: polimorfismo, herencia, encapsulamiento y abstracción.

3. Diseño orientado a objetos

3.1 Estructura del proyecto

```
Arreglo/  
├─ src/Arreglo/  
│   ├── AplicaArreglos.java      menú principal y método main  
│   ├── ArregloBase.java        clase abstracta: atributos, lectura y escritura  
│   ├── Arreglos.java           hereda de ArregloBase: eliminar, modificar, insertar y mover  
│   ├── ArregloEstadistico.java hereda de Arreglos e implementa Calculable: cálculos  
│   └─ Calculable.java          interfaz con los cálculos que se deben ofrecer
```

3.2 Diagrama de clases



3.3 Conceptos de POO aplicados

Concepto	Dónde se ve	Cómo se aplica
Encapsulamiento	<code>numerito</code> y <code>tope</code> en <code>ArregloBase</code>	Los atributos son <code>private</code> . Solo se accede a ellos con <code>getNumerito</code> , <code>setNumerito</code> , <code>getTope</code> y <code>setTope</code> . El setter de <code>tope</code> valida que esté entre 0 y la capacidad.
Herencia	<code>Arreglos</code> <code>extends</code> <code>ArregloBase</code> <code>ArregloEstadistico</code> <code>extends</code> <code>Arreglos</code>	Las clases hijas reutilizan atributos y métodos de sus padres sin repetir código. Es una cadena de tres niveles.
Abstracción	<code>ArregloBase</code> (clase abstracta) y la interfaz <code>Calculable</code>	<code>ArregloBase</code> declara <code>mostrarInformacion()</code> sin implementarlo y no se puede instanciar. <code>Calculable</code> define qué cálculos existen, sin decir cómo se hacen.
Polimorfismo (sobrescritura)	<code>mostrarInformacion()</code> en <code>Arreglos</code> y en <code>ArregloEstadistico</code> ; <code>escritura()</code> en <code>ArregloEstadistico</code>	Una variable de tipo <code>ArregloBase</code> puede apuntar a cualquier hija; al llamar al mismo método, responde la versión del objeto real. Se comprueba en la opción 12 del menú.
Polimorfismo (sobrecarga)	<code>insertar(x, y)</code> e <code>insertar(x)</code> ; los tres constructores	Mismo nombre con distintos parámetros: el compilador elige la versión según los argumentos.

3.4 Clases

ArregloBase

1. **Atributos privados.** `numerito` es el arreglo y `tope` cuenta cuántas casillas están realmente ocupadas. Como un arreglo tiene tamaño fijo, `tope` separa la parte usada de la libre; por eso todas las operaciones recorren hasta `tope` y no hasta `length`.
2. **Un solo Scanner.** `TECLADO` es `static` y lo comparten todas las clases, para no perder datos leídos por objetos distintos sobre `System.in`.
3. **Tres constructores.** Sin parámetros (arreglo sin crear), con tamaño (crea el arreglo y valida que sea mayor que 0) y de copia (duplica el contenido de otro arreglo).
4. **Getters y setters.** Único punto de acceso a los atributos; `setTope` lanza `IllegalArgumentException` si el valor no cabe.
5. **Consultas silenciosas.** `estaCreado()`, `tieneDatos()`, `estaLleno()` y `buscar(x)` devuelven un dato sin imprimir nada.
6. **Verificaciones con aviso.** `verificaCreado()`, `verificaConDatos()` y `verificaConEspacio()` devuelven `true` o `false` y le explican al usuario por qué no se puede continuar.
7. **Entrada segura.** `leerEntero(mensaje)` repite la pregunta hasta que el usuario escribe un entero válido.
8. **Lectura y escritura.** `lectura()` agrega un número en la posición `tope` y `escritura()` muestra el listado.

9. **Método abstracto.** mostrarInformacion() obliga a cada hija a describirse a su manera.

Arreglos

1. Hereda de ArregloBase; sus constructores llaman a super(...).
2. Agrega las operaciones que cambian el contenido: eliminar, modificar, insertar (con sobrecarga) y los dos movimientos.
3. Reutiliza buscar, los getters y las verificaciones del padre.
4. Implementa mostrarInformacion().

ArregloEstadistico

1. Hereda de Arreglos e implementa Calculable: cada método de la interfaz lleva @Override.
2. Los cálculos (sumar, hallarMenor, hallarMayor, hallarPromedio, sumarImpares) devuelven valores; los métodos mostrar... los presentan en pantalla.
3. Sobrescribe mostrarInformacion() y escritura() (esta última llama a super.escritura() y agrega la línea *Elementos: n de capacidad*).

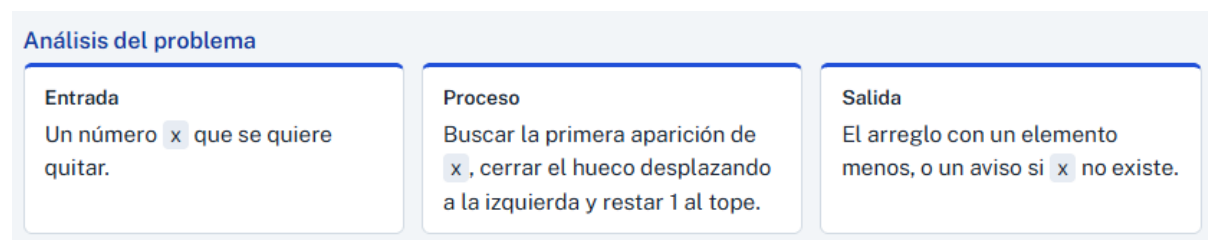
AplicaArreglos

1. Declara objArreglo de tipo ArregloEstadistico, que por herencia ya tiene todas las operaciones.
2. menu() repite un do...while hasta que se elige 0 y decide con un switch.
3. La opción 1 pide un tamaño mayor que 0 y crea el objeto con el constructor con parámetro.
4. Las opciones 4, 5 y 6 verifican primero (verificaConDatos() / verificaConEspacio()) y solo entonces piden los datos.
5. La opción 12 ejecuta demostracionPolimorfismo().

4. Desarrollo de las actividades

4.1 Eliminar un elemento x

Elaborar un método para eliminar un elemento "x", del arreglo creado anteriormente.



Algoritmo paso a paso

1. Comprobar que el arreglo exista y tenga datos.
2. Buscar la posición de x con buscar(x). Si devuelve -1, avisar y terminar.
3. Desde esa posición hasta la penúltima ocupada, copiar cada elemento desde su vecino de la derecha.
4. Limpiar la última casilla ocupada y disminuir el tope en 1.

Código

```
// 1. Eliminar un elemento "x"
public boolean eliminar(int x) {
    if (!verificaConDatos()) {
        return false;
    }

    int pos = buscar(x);
    if (pos == -1) {
        System.out.println("El elemento " + x + " no se encuentra en el arreglo.");
        return false;
    }

    int[] a = getNumerito();
    for (int i = pos; i < getTope() - 1; i++) {
        a[i] = a[i + 1]; // se corren los elementos a la izquierda
    }
    a[getTope() - 1] = 0;
    setTope(getTope() - 1);
    System.out.println("Elemento " + x + " eliminado correctamente.");
    return true;
}
```

Explicación del código

1. verificaConDatos() evita trabajar con un arreglo sin crear o vacío: muestra el aviso y el método devuelve false.
2. buscar(x) es un método heredado de ArregloBase. Recorre de 0 a tope - 1 y devuelve la posición de la primera aparición, o -1.
3. El for que empieza en pos hace a[i] = a[i + 1]: cada elemento sobrescribe al anterior, así el hueco de x se cierra y todo queda una casilla más a la izquierda.
4. a[getTope() - 1] = 0 limpia la última casilla, que quedó repetida tras el desplazamiento.
5. setTope(getTope() - 1) reduce la cantidad de elementos válidos. El setter comprueba que el valor esté entre 0 y la capacidad.

Ejemplo con los datos de prueba



Evidencia de ejecución

```

Listado
-----
5      12      7      20      3      8      15
Elementos: 7 de 8

```

Ingrese una Alternativa : 4

Ingrese el elemento a eliminar : 20
Elemento 20 eliminado correctamente.

Ingrese una Alternativa : 3

```

Listado
-----
5      12      7      3      8      15
Elementos: 6 de 8

```

Resultado

Datos iniciales	Entrada del usuario	Resultado esperado	Resultado obtenido	¿Coincide?
[5, 12, 7, 20, 3, 8, 15]	Opción 4, x = 20	[5, 12, 7, 3, 8, 15]	[5, 12, 7, 3, 8, 15]	Sí

4.2 Modificar un elemento x

Elaborar un método para modificar un elemento "x".

Análisis del problema

Entrada

El elemento `x` a buscar y el valor `nuevo`.

Proceso

Localizar `x` y reemplazarlo por `nuevo` sin mover nada más.

Salida

El arreglo con ese valor cambiado, o un aviso si `x` no existe.

Algoritmo paso a paso

1. Comprobar que el arreglo exista y tenga datos.
2. Buscar la posición de x. Si no está, avisar y terminar.
3. Escribir nuevo en esa posición.

Código

```
// 2. Modificar un elemento "x" por un valor nuevo
public boolean modificar(int x, int nuevo) {
    if (!verificaConDatos()) {
        return false;
    }

    int pos = buscar(x);
    if (pos == -1) {
        System.out.println("El elemento " + x + " no se encuentra en el arreglo.");
        return false;
    }

    getNumerito()[pos] = nuevo;
    System.out.println("Elemento " + x + " modificado por " + nuevo + ".");
    return true;
}
```

Explicación del código

1. El menú pide dos datos: el elemento que se busca (x) y el valor que lo reemplaza (nuevo).
2. buscar(x) devuelve la posición; con -1 no se modifica nada.
3. getNumerito()[pos] = nuevo reemplaza el valor en su lugar. No se mueve ningún otro elemento y el tope no cambia.
4. Si x aparece repetido, solo se modifica la primera aparición.

Ejemplo con los datos de prueba



Evidencia de ejecución

```

Listado
-----
5      12      7      20      3      8      15
Elementos: 7 de 8

```

Ingrese una Alternativa : 5

Ingrese el elemento a modificar : 7

Ingrese el nuevo valor : 50

Elemento 7 modificado por 50.

Ingrese una Alternativa : 3

```

Listado
-----
5      12      50      20      3      8      15
Elementos: 7 de 8

```

Resultado

Datos iniciales	Entrada del usuario	Resultado esperado	Resultado obtenido	¿Coincide?
[5, 12, 7, 20, 3, 8, 15]	Opción 5, x = 7, nuevo = 50	[5, 12, 50, 20, 3, 8, 15]	[5, 12, 50, 20, 3, 8, 15]	Sí

4.3 Insertar un elemento x en una posición y

Análisis del problema

Entrada

El elemento `x` y la posición `y` (contada desde 1).

Proceso

Abrir un hueco desplazando a la derecha desde la última casilla hasta `y` y colocar `x`.

Salida

El arreglo con un elemento más, o un aviso si está lleno o la posición no es válida.

Algoritmo paso a paso

1. Comprobar que el arreglo exista y que tenga espacio libre.
2. Validar que `y` esté entre 1 y `tope + 1`.
3. Recorrer de derecha a izquierda desde `tope` hasta `y`, copiando cada elemento una casilla a la derecha.
4. Escribir `x` en la casilla `y - 1` y aumentar el `tope` en 1.

Código:

```
// 3. Insertar un elemento "x" en la posicion "y"
public boolean insertar(int x, int y) {
    if (!verificaConEspacio()) {
        return false;
    }
    if (y < 1 || y > getTope() + 1) {
        System.out.println("Posicion invalida. Debe estar entre 1 y " + (getTope() + 1) + ".");
        return false;
    }

    int[] a = getNumerito();
    for (int i = getTope(); i > y - 1; i--) {
        a[i] = a[i - 1];        // se corren los elementos a la derecha
    }
    a[y - 1] = x;
    setTope(getTope() + 1);
    System.out.println("Elemento " + x + " insertado en la posicion " + y + ".");
    return true;
}
```

Explicación del código

1. La posición `y` se pide contando desde 1, como lo haría una persona. En el arreglo esa casilla es `y - 1`.
2. `verificaConEspacio()` confirma que exista una casilla libre (`tope < length`).
3. La condición `y < 1 || y > getTope() + 1` rechaza posiciones fuera de rango. `tope + 1` equivale a insertar al final.
4. El `for` va de derecha a izquierda (`i--`) y hace `a[i] = a[i - 1]`. Se recorre en ese sentido para no pisar datos que todavía no se han movido.
5. `a[y - 1] = x` ocupa el hueco y `setTope(getTope() + 1)` registra el nuevo elemento.
6. **Sobrecarga:** `insertar(int x)` tiene el mismo nombre pero un solo parámetro; llama a `insertar(x, getTope() + 1)` para agregar al final.

Ejemplo con los datos de prueba



Evidencia de ejecución

```
Listado
-----
5      12      7      20      3      8      15
Elementos: 7 de 8
```

Ingrese una Alternativa : 6

```
Ingrese el elemento a insertar : 99
Ingrese la posicion (1 a 8) : 3
Elemento 99 insertado en la posicion 3.
```

```
Listado
-----
5      12      99      7      20      3      8      15
Elementos: 8 de 8
```

Resultado

Datos iniciales	Entrada del usuario	Resultado esperado	Resultado obtenido	¿Coincide?
[5, 12, 7, 20, 3, 8, 15]	Opción 6, x = 99, y = 3	[5, 12, 99, 7, 20, 3, 8, 15]	[5, 12, 99, 7, 20, 3, 8, 15]	Sí

4.4 Sumar los elementos del arreglo

Elaborar un método para sumar los elementos del arreglo.

Análisis del problema

Entrada

Ninguna: usa los elementos ya ingresados.

Proceso

Acumular cada elemento en una variable que empieza en 0.

Salida

Un número entero con la suma total.

Algoritmo paso a paso

1. Comprobar que el arreglo tenga datos.
2. Iniciar suma = 0.
3. Recorrer de 0 a tope - 1 sumando cada elemento.
4. Devolver suma y mostrarla.

Código

```
private void exigeDatos() {
    if (!tieneDatos()) {
        throw new IllegalStateException("El arreglo no tiene elementos.");
    }
}

// 4. Sumar los elementos del arreglo
@Override
public int sumar() {
    exigeDatos();
    int suma = 0;
    for (int i = 0; i < getTope(); i++) {
        suma += getNumerito()[i];
    }
    return suma;
}

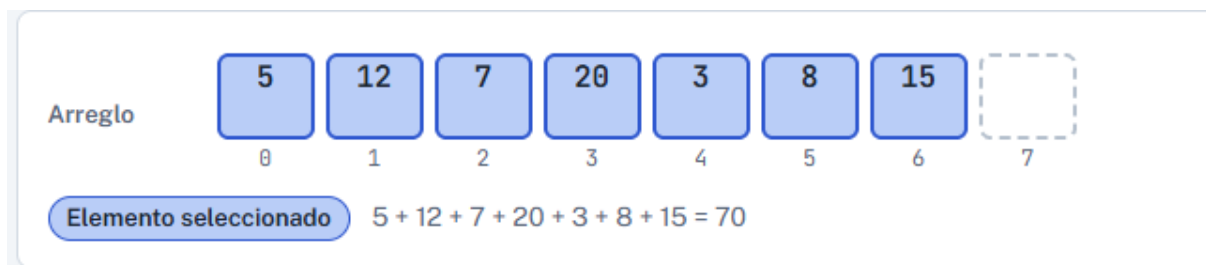
public void mostrarSuma() {
    if (!verificaConDatos()) {
        return;
    }
    System.out.println("La suma de los elementos es: " + sumar());
}
```

Explicación del código

1. sumar() está declarado en la interfaz Calculable (qué se debe hacer) y se implementa en ArregloEstadistico (cómo se hace).
2. exigeDatos() lanza IllegalStateException si el arreglo está vacío, para que un cálculo nunca trabaje con datos que no existen.
3. El acumulador suma empieza en 0. El for llega hasta tope, no hasta length: las casillas libres no cuentan.

4. mostrarSuma() separa la presentación del cálculo: primero verifica con un mensaje amable y después imprime el resultado que devuelve sumar().

Ejemplo con los datos de prueba



Evidencia de ejecución

```
Listado
-----
5      12      7      20      3      8      15
Elementos: 7 de 8

Ingrese una Alternativa : 7

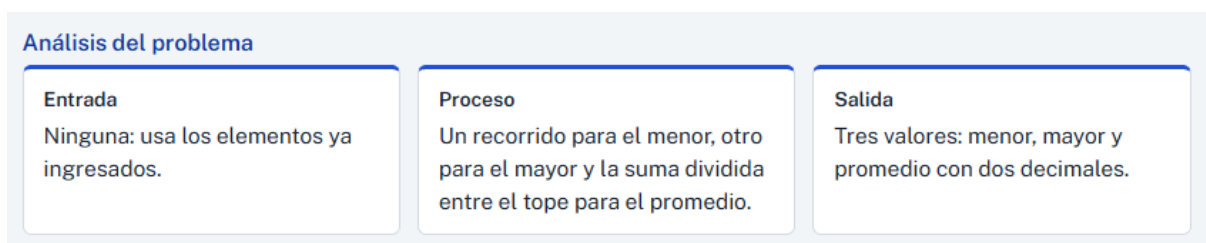
La suma de los elementos es: 70
```

Resultado

Datos iniciales	Entrada del usuario	Resultado esperado	Resultado obtenido	¿Coincide?
[5, 12, 7, 20, 3, 8, 15]	Opción 7	70	70	Sí

4.5 Número menor, número mayor y promedio

Elaborar un método para hallar el número menor, el número mayor y el promedio de los elementos del arreglo.



Algoritmo paso a paso

1. Comprobar que el arreglo tenga datos.

2. Menor: tomar el primer elemento como candidato y compararlo con los demás; quedarse con el más pequeño.
3. Mayor: igual, quedándose con el más grande.
4. Promedio: dividir la suma total entre la cantidad de elementos (tope).
5. Mostrar los tres resultados.

Código

```
// 5a. Numero menor
@Override
public int hallarMenor() {
    exigeDatos();
    int menor = getNumerito()[0];
    for (int i = 1; i < getTope(); i++) {
        if (getNumerito()[i] < menor) {
            menor = getNumerito()[i];
        }
    }
    return menor;
}

// 5b. Numero mayor
@Override
public int hallarMayor() {
    exigeDatos();
    int mayor = getNumerito()[0];
    for (int i = 1; i < getTope(); i++) {
        if (getNumerito()[i] > mayor) {
            mayor = getNumerito()[i];
        }
    }
    return mayor;
}

// 5c. Promedio
@Override
public double hallarPromedio() {
    return (double) sumar() / getTope();
}

public void mostrarEstadisticas() {
    if (!verificaConDatos()) {
        return;
    }
    System.out.println("Numero menor : " + hallarMenor());
    System.out.println("Numero mayor : " + hallarMayor());
    System.out.println("Promedio      : " + String.format(Locale.US, "%.2f", hallarPromedio()));
}
```

Explicación del código

1. menor y mayor empiezan con el primer elemento, no con 0. Si empezaran en 0, un arreglo con todos los números negativos daría un mayor incorrecto.
2. El for arranca en $i = 1$ porque la posición 0 ya es el candidato inicial.
3. (double) sumar() / getTope(): el cast a double evita la división entera, que descartaría los decimales.
4. String.format(Locale.US, "%.2f", ...) muestra el promedio con dos decimales y punto decimal.
5. Cada método es independiente (una sola responsabilidad); mostrarEstadisticas() los combina para el menú.

Ejemplo con los datos de prueba



Evidencia de ejecución

```

Listado
-----
5      12      7      20      3      8      15
Elementos: 7 de 8

Ingrese una Alternativa : 8

Numero menor : 3
Numero mayor : 20
Promedio      : 10.00

```

Resultado

Datos iniciales	Entrada del usuario	Resultado esperado	Resultado obtenido	¿Coincide?
[5, 12, 7, 20, 3, 8, 15]	Opción 8	menor 3, mayor 20, promedio 10.00	menor 3, mayor 20, promedio 10.00	Sí

4.6 Suma de los elementos impares

Elaborar un método para suma de Impares los elementos del arreglo.

Análisis del problema

Entrada

Ninguna: usa los elementos ya ingresados.

Proceso

Recorrer el arreglo y acumular solo los elementos impares.

Salida

Un número entero con la suma de los impares.

Algoritmo paso a paso

1. Comprobar que el arreglo tenga datos.
2. Iniciar suma = 0.
3. Recorrer de 0 a tope - 1; si el elemento es impar, sumarlo.
4. Devolver suma y mostrarla.

Código

```
// 6. Suma de los elementos impares
@Override
public int sumarImpares() {
    exigeDatos();
    int suma = 0;
    for (int i = 0; i < getTope(); i++) {
        if (getNumerito()[i] % 2 != 0) { // funciona tambien con negativos
            suma += getNumerito()[i];
        }
    }
    return suma;
}
```

Explicación del código

1. La prueba de imparidad es elemento % 2 != 0.
2. Se usa != 0 y no == 1 porque en Java el residuo de un impar negativo es -1 (por ejemplo, -3 % 2 vale -1). Con == 1 los impares negativos no se sumarían.
3. El resto es igual que en la suma total, pero con la condición if como filtro.

Ejemplo con los datos de prueba

Arreglo

5	12	7	20	3	8	15	
0	1	2	3	4	5	6	7

Elemento seleccionado 5 + 7 + 3 + 15 = 30

Evidencia de ejecución

Ingrese una Alternativa : 3

```

      Listado
      -----
5      12      7      20      3      8      15
Elementos: 7 de 8

```

Ingrese una Alternativa : 9

La suma de los elementos impares es: 30

Resultado

Datos iniciales	Entrada del usuario	Resultado esperado	Resultado obtenido	¿Coincide?
[5, 12, 7, 20, 3, 8, 15]	Opción 9	30	30	Sí

4.7 Mover el primer elemento a la última posición

Elaborar un método para mover el Primer elemento a la Última posición del arreglo.

Análisis del problema

Entrada

Ninguna: usa los elementos ya ingresados.

Proceso

Guardar el primer elemento, desplazar todo a la izquierda y colocarlo al final.

Salida

El mismo arreglo con el primer elemento en la última posición ocupada.

Algoritmo paso a paso

1. Comprobar que el arreglo tenga datos.
2. Guardar el primer elemento en una variable auxiliar.
3. Desplazar cada elemento una casilla a la izquierda.
4. Escribir el elemento guardado en la última posición ocupada.

Código

```
// 7. Mover el primer elemento a la ultima posicion
public void moverPrimeroAlUltimo() {
    if (!verificaConDatos()) {
        return;
    }

    int[] a = getNumerito();
    int primero = a[0];
    for (int i = 0; i < getTope() - 1; i++) {
        a[i] = a[i + 1];
    }
    a[getTope() - 1] = primero;
    System.out.println("El primer elemento (" + primero + ") paso a la ultima posicion.");
}
```

Explicación del código

1. La variable primero guarda el valor de a[0]; sin ella se perdería al desplazar.
2. El for va de 0 a tope - 2 y hace a[i] = a[i + 1].
3. El valor guardado se escribe en a[getTope() - 1] (última casilla ocupada) y no en length - 1, porque puede haber casillas libres al final.
4. Con un solo elemento el resultado no cambia, y el método funciona sin error.

Ejemplo con los datos de prueba



Evidencia de ejecución

Ingrese una Alternativa : 3

```

Listado
-----
5      12      7      20      3      8      15
Elementos: 7 de 8
```

Ingrese una Alternativa : 10

El primer elemento (5) paso a la ultima posicion.

```

Listado
-----
12      7      20      3      8      15      5
Elementos: 7 de 8
```

Resultado

Datos iniciales	Entrada del usuario	Resultado esperado	Resultado obtenido	¿Coincide?
[5, 12, 7, 20, 3, 8, 15]	Opción 10	[12, 7, 20, 3, 8, 15, 5]	[12, 7, 20, 3, 8, 15, 5]	Sí

4.8 Mover el primer elemento par al final

Elaborar un método para mover el Primer elemento Par al Final del arreglo.

Análisis del problema

Entrada

Ninguna: usa los elementos ya ingresados.

Proceso

Buscar el primer par, desplazar a la izquierda desde su posición y colocarlo al final.

Salida

El arreglo con ese par al final, o un aviso si no hay pares.

Algoritmo paso a paso

1. Comprobar que el arreglo tenga datos.
2. Recorrer buscando el primer elemento par y guardar su posición.
3. Si no hay pares, avisar y terminar.
4. Guardar el par, desplazar a la izquierda desde su posición y colocarlo en la última posición ocupada.

Código

```
// 8. Mover el primer elemento par al final del arreglo
public void moverPrimerParAlFinal() {
    if (!verificaConDatos()) {
        return;
    }

    int[] a = getNumerito();
    int pos = -1;
    for (int i = 0; i < getTope(); i++) {
        if (a[i] % 2 == 0) {
            pos = i;
            break;
        }
    }

    if (pos == -1) {
        System.out.println("No hay elementos pares en el arreglo.");
        return;
    }

    int par = a[pos];
    for (int i = pos; i < getTope() - 1; i++) {
        a[i] = a[i + 1];
    }
    a[getTope() - 1] = par;
    System.out.println("El primer par (" + par + ") paso al final del arreglo.");
}
}
```

Explicación del código

1. El primer for busca la primera casilla con $a[i] \% 2 == 0$. El break lo detiene apenas la encuentra, así se garantiza que sea el primer par.
2. $pos == -1$ significa que no hay pares (el 0 sí cuenta como par). Se muestra un aviso y el arreglo queda igual.
3. Se guarda el valor en par, el segundo for desplaza a la izquierda desde pos y el valor se escribe en $a[getTope() - 1]$.
4. Es la actividad 7 con un paso extra: en la 7 el elemento a mover siempre está en la posición 0; aquí primero hay que encontrarlo.

Ejemplo con los datos de prueba



Evidencia de ejecución

```

Listado
-----
5      12      7      20      3      8      15
Elementos: 7 de 8

```

Ingrese una Alternativa : 11

El primer par (12) paso al final del arreglo.

Ingrese una Alternativa : 3

```

Listado
-----
5      7      20      3      8      15      12
Elementos: 7 de 8

```

Resultado

Datos iniciales	Entrada del usuario	Resultado esperado	Resultado obtenido	¿Coincide?
[5, 12, 7, 20, 3, 8, 15]	Opción 11	[5, 7, 20, 3, 8, 15, 12]	[5, 7, 20, 3, 8, 15, 12]	Sí

5. Conclusiones

- Un arreglo tiene tamaño fijo. El atributo tope permite saber cuántas casillas están ocupadas, y por eso los recorridos llegan hasta tope - 1.
- Insertar y eliminar obligan a desplazar elementos, y el sentido del recorrido importa: de derecha a izquierda al insertar y de izquierda a derecha al eliminar, para no pisar datos.

- **Separar el código en una clase base abstracta, una clase con las operaciones de edición y otra con los cálculos dejó métodos cortos, con una sola responsabilidad y fáciles de probar.**
- **La interfaz `Calculable` y el método abstracto `mostrarInformacion()` muestran la abstracción; la opción 12 del menú comprueba el polimorfismo.**
- **Validar las entradas (arreglo sin crear, lleno, posiciones inválidas, texto en lugar de números) evita errores en tiempo de ejecución.**